МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность
Образовательная программа: «Технологии защиты информации»

Дисциплина:
«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Разработка программного обеспечения с механизмами контроля лимитов пробного использования (Trial-версия) и защитой от переустановки»

Выполнил студент:

Пахомов Владимир Юрьевич / 

ФИО *Подпись*

Проверил:

Ищенко Алексей Петрович / _____

ФИО *Подпись*

_____

Отметка о выполнении

_____

Дата

Санкт-Петербург
2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Архитектура и алгоритмы системы защиты ПО **4**
1.1 Общая структура программного комплекса 4
1.2 Структура хранения лицензионных данных 4
1.3 Алгоритм контроля ограничений 4

2 Реализация программных компонентов **5**
2.1 Инсталлятор и обнаружение повторной установки 5
2.2 Основное приложение 5
2.3 Деинсталлятор 5

3 Анализ защищенности и сценарии обхода защиты **6**
3.1 Точки регистрации в операционной системе 6
3.2 Сценарии несанкционированного сброса ограничений («взлом») 6
3.3 Рекомендации по усилению устойчивости защиты 6

4 Руководство пользователя **8**
4.1 Установка программы 8
4.2 Эксплуатация программы 8
4.3 Действия при истечении ознакомительного периода 9
4.4 Деинсталляция 9

ЗАКЛЮЧЕНИЕ **11**

ПРИЛОЖЕНИЕ. ЛИСТИНГИ ИСХОДНОГО КОДА **12**

ВВЕДЕНИЕ

Цель работы: приобретение практических навыков реализации прикладных механизмов защиты программных продуктов от несанкционированного тиражирования и неправомерного продления ознакомительного периода, разработка программного комплекса (инсталлятор, приложение, деинсталлятор) с комбинированным ограничением по времени и числу запусков, а также проведение анализа защищенности системы и способов ее обхода («взлома»).

Постановка задачи:

1. Разработать программу, запрашивающую у пользователя фамилию, имя и отчество (ФИО) и сохраняющую их в файл профиля. При совпадении введенных данных с сохраненными ранее выводить соответствующее сообщение.
2. Реализовать две версии программы (или одну совмещенную):
    - *Time-limited*: ограничение по времени жизненного цикла ознакомительного режима с момента первого старта (не более 3 минут).
    - *Start-limited*: жесткое ограничение по количеству запусков приложения (например 4-5 запусков);
3. При исчерпании любого из лимитов блокировать ввод данных и предлагать пользователю два действия: переход к приобретению полной лицензии либо запуск встроенного деинсталлятора.
4. Разработать графический инсталлятор, регистрирующий ПО в среде пользователя.
5. Обеспечить сохранение истории лицензирования: при повторной установке инсталлятор обязан детектировать факт прошлой инсталляции, информировать пользователя об израсходованном лимите и запрещать сброс счетчиков.
6. Разработать деинсталлятор, корректно очищающий исполняемые бинарные файлы и ярлыки рабочего окружения, но сохраняющий скрытый маркер лицензирования.

1 АРХИТЕКТУРА И АЛГОРИТМЫ СИСТЕМЫ ЗАЩИТЫ ПО

1.1 Общая структура программного комплекса

Программный комплекс спроектирован по модульному принципу и ориентирован на исполнение в современных дистрибутивах Linux с поддержкой стандартов XDG Base Directory Specification и графического окружения KDE Plasma.

Комплекс включает три ключевых компонента:

1. **Модуль установки (`installer.py`)**: выполняет проверку предыдущих установок, развертывает файлы в системные пути пользователя, создает `.desktop`-файлы для интеграции в меню программ и на Рабочий стол, а также валидирует доверенные атрибуты запуска через подсистему GIO;
2. **Основное приложение (`main.py`)**: графическая утилита на базе Tkinter, реализующая логику проверки лицензионных ограничений, обработку ФИО и выполнение блокировки функционала;
3. **Модуль деинсталляции (`uninstaller.py`)**: осуществляет безопасное удаление исполняемых компонентов программы и интеграций с рабочим столом, сохраняя лицензионный артефакт в каталоге конфигураций пользователя.

1.2 Структура хранения лицензионных данных

В соответствии со спецификацией XDG, конфигурационный каталог пользователя располагается по пути `~/.config/lab2/`. В нем создается структурированный текстовый файл `lab2.txt`. Структура файла описана в таблице 1.

Таблица 1 — Спецификация строк файла конфигурации и лицензии `lab2.txt`

№ строки	Поле	Тип данных	Назначение
1	`name`	Текстовая строка (UTF-8)	Хранение зарегистрированного ФИО пользователя
2	`launches`	Целое неотрицательное	Счетчик совершенных запусков приложения (Start-limit)
3	`first_launch`	Строка ISO-8601 UTC	Временная метка первого запуска программы (Time-limit)

1.3 Алгоритм контроля ограничений

Приложение сочетает в себе два независимых механизма контроля: по количеству запусков и по времени с момента первого сохранения ФИО. При каждом старте выполняется проверка условий, и если хотя бы одно из них нарушено, интерфейс блокируется, предлагая пользователю деинсталлировать программу или связаться с поддержкой. Под запуском программы подразумевается сохранение ФИО в файл: именно тогда счетчик увеличивается на единицу, а временная метка фиксируется при первом сохранении.

2 РЕАЛИЗАЦИЯ ПРОГРАММНЫХ КОМПОНЕНТОВ

2.1 Инсталлятор и обнаружение повторной установки

Инсталлятор (`installer.py`) реализует полный цикл развертывания приложения без необходимости прав суперпользователя (`root`), используя домашний каталог пользователя согласно спецификациям freedesktop.org:

- Исполняемые файлы копируются в директорию `~/.local/share/lab2/`;
- Интеграция в системное меню приложений рабочего окружения выполняется путем генерации `.desktop` файла в `~/.local/share/applications/lab2.desktop`;
- Автоматически определяется каталог Рабочего стола с запасными путями (`~/Desktop`, `~/Рабочий стол`) и создается ярлык запуска.

Перед началом копирования файлов процедура `check_previous_install()` обращается к файлу `~/.config/lab2/lab2.txt`. Если файл существует и в строке запусков зафиксировано ненулевое значение, инсталлятор выводит предупреждающее сообщение (оранжевый текст в GUI) о том, что система уже находилась на данном компьютере, и сброса лимитов не произойдет.

2.2 Основное приложение

Модуль `main.py` управляет жизненным циклом программы:

1. Функция `read_conf()` парсит конфигурационный файл. В случае первого старта генерируется временная метка `first_launch` в часовом поясе UTC, фиксируемая через `datetime.timezone.utc`.
2. Если лимиты не превышены, интерфейс отображает текущее сохраненное ФИО (если оно было записано), поле `Entry` для ввода новых данных и подсказка с оставшимся числом запусков.
3. При нажатии на кнопку «Сохранить» счетчик запусков увеличивается на единицу, данные сериализуются в файл конфигурации, и выполняется повторная проверка условий.
4. Если условие `is_trial_expired()` истинно, интерфейс переводится в режим блокировки: поле ввода и кнопка сохранения отключаются (`state="disabled"`), динамически генерируется ссылка на приобретение лицензии и появляется кнопка перехода к деинсталляции.

2.3 Деинсталлятор

Модуль `uninstaller.py` отвечает за корректную деинсталляцию:

- Спрашивает у пользователя подтверждение удаления через диалоговое окно `messagebox.as`
- Удаляет исполняемую директорию `~/.local/share/lab2/`;
- Удаляет файлы `.desktop` из меню и с Рабочего стола;
- **Ключевая особенность защиты**: конфигурационный файл `~/.config/lab2/lab2.txt` сознательно остается на накопителе. Так гарантируется учет использованных лимитов при повторной установке.

3 АНАЛИЗ ЗАЩИЩЕННОСТИ И СЦЕНАРИИ ОБХОДА ЗАЩИТЫ

Согласно техническому заданию, в рамках лабораторной работы необходимо четко идентифицировать места регистрации программы в операционной системе и определить векторы атак («взлома»), позволяющие злоумышленнику сбросить ограничения.

3.1 Точки регистрации в операционной системе

В процессе функционирования ПО создает в ОС следующие артефакты:

1. **Каталог исполняемых файлов**: `$HOME/.local/share/lab2/main`, `$HOME/.local/share/lab2/uninstaller`;
2. **Файл пункта меню KDE**: `$HOME/.local/share/applications/lab2.desktop`;
3. **Ярлык на рабочем столе**: `$HOME/Desktop/lab2.desktop`;
4. **Файл состояния лицензии**: `$HOME/.config/lab2/lab2.txt`.

3.2 Сценарии несанкционированного сброса ограничений («взлом»)

Поскольку контрольные маркеры хранятся в открытом текстовом формате, существуют следующие тривиальные и эффективные методы обхода ограничений со стороны недобросовестного пользователя:

1. **Удаление лицензионного файла**: Полное удаление файла артефакта приводит систему в состояние «первичной установки»:

```
rm -rf ~/.config/lab2
```

   После выполнения данной команды инсталлятор и основная программа воспринимают сессию как абсолютно новую, выделяя 3 новых запуска и 1 минуту пробного периода.
2. **Фальсификация значений счетчика запусков и временной метки**: Пользователь может вручную скорректировать вторую строку файла `lab2.txt`, установив значение запусков в 0 или даже отрицательное число, а также обновить дату в третьей строке на будущую:
3. **Заморозка файла от модификаций**: С помощью утилиты изменения файловых атрибутов `chattr` пользователь может запретить приложению обновлять файл:

```
chmod 444 ~/.config/lab2/lab2.txt
```

   В этом случае при попытке записи программа вызовет исключение, либо (при отсутствии обработки) счетчик не будет увеличиваться.

3.3 Рекомендации по усилению устойчивости защиты

Для перевода механизма в категорию промышленной защиты от несанкционированного тиражирования рекомендуется:

- Использовать криптографическое хеширование содержимого конфигурационного файла с солью, привязанной к уникальным аппаратным характеристикам компьютера (серий-

ный номер материнской платы, UUID корневого раздела диска);

- Распределять метки первого старта по нескольким скрытым местам ОС;

4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

4.1 Установка программы

Для первичного или повторного развертывания программного продукта запустите графический установщик командой:

```
python3 installer.py
```

Или, если инсталлятор упакован в бинарный файл через PyInstaller:

```
./installer
```

При первичной или повторной установке появится соответствующий текст. После нажатия кнопки «Установить» отобразится модальное окно об успешной регистрации ярлыков (Рисунок 1 – 2).

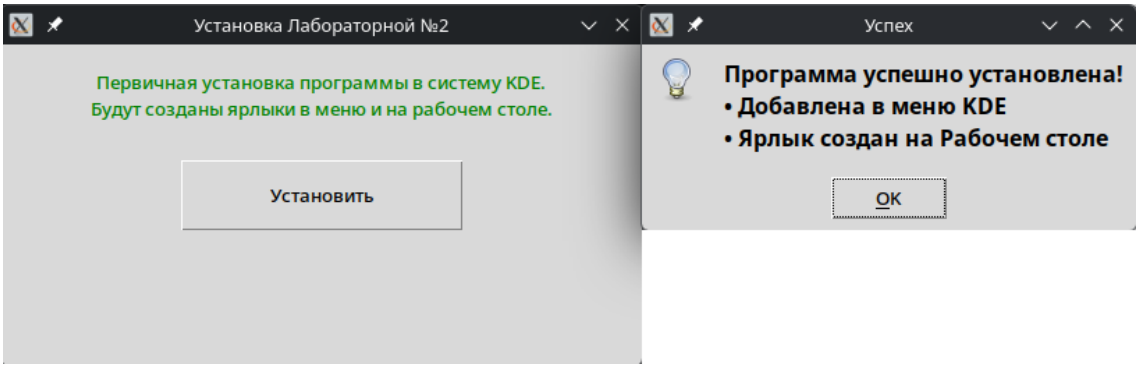


Рисунок 1 — Диалоговое окно успешной установки программы в систему

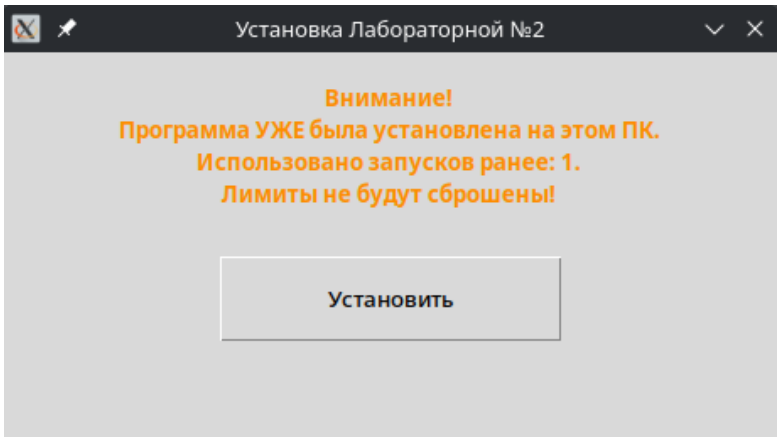


Рисунок 2 — Диалоговое окно предупреждения о повторной установке программы с сохранением лимитов

4.2 Эксплуатация программы

Запуск приложения доступен как через иконку «Лабораторная 2 (Trial)» в главном меню приложений рабочего окружения или на Рабочем столе, так и из терминала:

```
~/.local/share/lab2/main
# либо в среде разработки
python3 main.py
```

При открытии главного окна программы (Рисунок 3) пользователь видит:

- В верхней части отображается ранее сохраненное ФИО (если запись уже производилась) либо приглашение к вводу;
- В нижней части окна отображается статус триального периода: «Бесплатных запусков осталось: N»;
- Введите ФИО и нажмите кнопку «Сохранить». Данные запишутся на диск, а счетчик оставшихся попыток уменьшится.

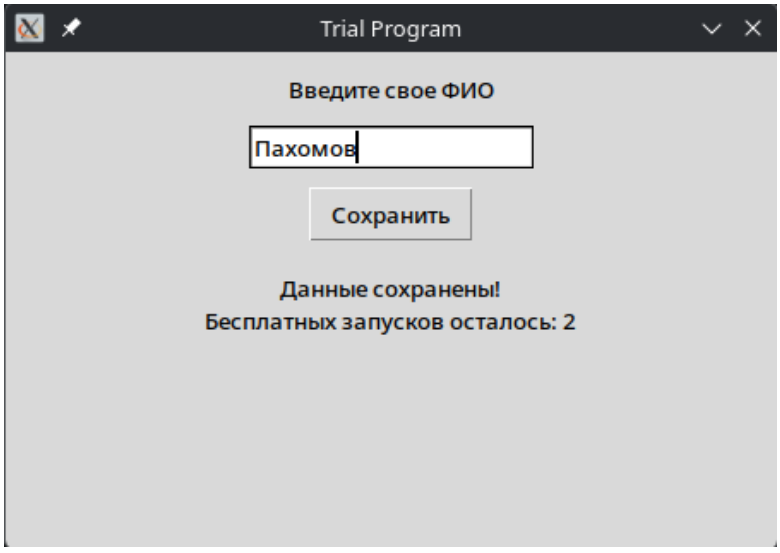


Рисунок 3 — Главное окно программы с полем ввода ФИО и информацией о лимите запусков

4.3 Действия при истечении ознакомительного периода

После совершения 3 запусков либо по истечении 1 минуты с момента первого включения:

1. Поля ввода блокируются;
2. Отображается красный статус ошибки («Вы превысили лимит запусков программы» либо «Вы превысили лимит времени использования программы»);
3. Предоставляется кликабельная гиперссылка для связи с разработчиком;
4. Появляется красная кнопка «Удалить программу (Деинсталляция)».
   Интерфейс блокировки показан на Рисунке 4.

4.4 Деинсталляция

При нажатии на кнопку удаления или ручном запуске модуля `uninstaller.py`:

1. Подтвердите согласие в диалоговом окне подтверждения;
2. Программа удалит бинарные файлы и ярлыки, уведомив об успешном завершении операции.

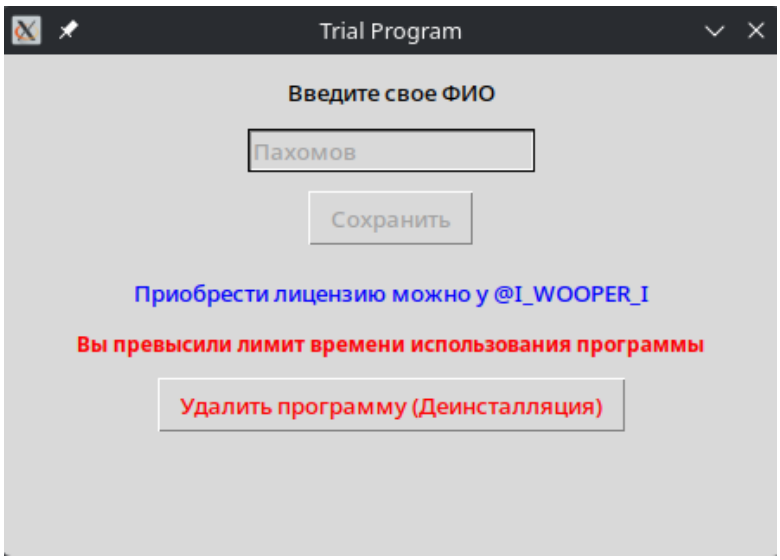


Рисунок 4 — Главное окно программы в режиме блокировки после исчерпания лимитов

Интерфейс деинсталлятора показан на Рисунке 5.

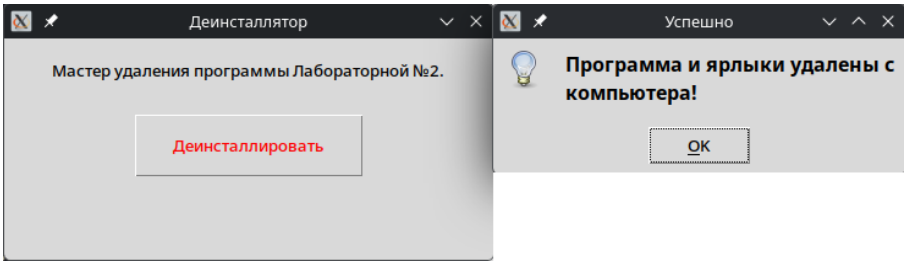


Рисунок 5 — Диалоговое окно деинсталлятора с подтверждением удаления программы

ЗАКЛЮЧЕНИЕ

В процессе выполнения лабораторной работы №2 был спроектирован, реализован и всесторонне протестирован программный комплекс защиты условно-бесплатного приложения (Trial-версии) от несанкционированного тиражирования и неправомерного продления пробного периода.

Основные результаты работы:

1. Разработан графический инсталлятор (`installer.py`), интегрирующий приложение в рабочее окружение Linux/KDE через спецификации freedesktop/XDG и осуществляющий детектирование предыдущих установок продукта на целевой рабочей станции.
2. Реализована основная программа (`main.py`) с графическим интерфейсом на базе Tkinter, обеспечивающая сбор, хранение и валидацию ФИО пользователя.
3. Реализован комбинированный механизм защиты с параллельным контролем двух метрик: лимита на суммарное число запусков (Start-limited) и временного окна использования с момента первого старта (Time-limited).
4. Разработан деинсталлятор (`uninstaller.py`), удаляющий исполняемые файлы и ярлыки рабочего стола, но сохраняющий системный скрытый артефакт в каталоге конфигураций, что исключает сброс лимитов при переустановке.
5. Проведен анализ слабых мест реализованной архитектуры хранения лицензий и сформулированы практические рекомендации по повышению стойкости защиты от реверс-инжиниринга и локального взлома.

ПРИЛОЖЕНИЕ. ЛИСТИНГИ ИСХОДНОГО КОДА

Листинг 1. Модуль инсталлятора (installer.py)

```
import os
import sys
import shutil
import subprocess
from tkinter import Tk, Label, Button, messagebox

def get_resource_path():
    """Получение пути к ресурсам при упаковке через PyInstaller"""
    if getattr(sys, 'frozen', False) and hasattr(sys, '_MEIPASS'):
        return sys._MEIPASS
    return os.path.dirname(os.path.abspath(__file__))

def get_desktop_dir():
    """Определение пути к каталогу Рабочего стола в соответствии со стандартами
    XDG"""
    try:
        res = subprocess.check_output(["xdg-user-dir", "DESKTOP"], text=True).
    strip()
        if os.path.exists(res):
            return res
    except Exception:
        pass

    for fallback in [os.path.expanduser("~/Desktop"), os.path.expanduser("
    Рабочий~/ стол")]:
        if os.path.exists(fallback):
            return fallback
    return os.path.expanduser("~/Desktop")

BUNDLE_DIR = get_resource_path()
HOME = os.path.expanduser("~")

INSTALL_DIR = os.path.join(HOME, ".local", "share", "lab2")
APPS_DIR = os.path.join(HOME, ".local", "share", "applications")
DESKTOP_DIR = get_desktop_dir()

MENU_DESKTOP_FILE = os.path.join(APPS_DIR, "lab2.desktop")
DESKTOP_SHORTCUT_FILE = os.path.join(DESKTOP_DIR, "lab2.desktop")
CONF_FILE = os.path.join(HOME, ".config", "lab2", "lab2.txt")

def check_previous_install():
    """Проверка наличия следов предыдущей инсталляции и вычитка счетчика"""
    if os.path.exists(CONF_FILE):
        try:
            with open(CONF_FILE, "r") as f:
```

```
                lines = f.read().splitlines()
                if len(lines) >= 2 and int(lines[1]) > 0:
                    return True, int(lines[1])
        except Exception:
            pass
    return False, 0

def install():
    """Процедура развертывания исполняемых модулей и ярлыков в системе"""
    try:
        os.makedirs(INSTALL_DIR, exist_ok=True)
        os.makedirs(APPS_DIR, exist_ok=True)
        os.makedirs(DESKTOP_DIR, exist_ok=True)

        for filename in ["main", "uninstaller"]:
            src = os.path.join(BUNDLE_DIR, filename)
            if not os.path.exists(src):
                raise FileNotFoundError(f"Вшитый файл {filename} не найден!")

            dst = os.path.join(INSTALL_DIR, filename)
            shutil.copy2(src, dst)
            os.chmod(dst, 0o755)

        exec_path = os.path.join(INSTALL_DIR, "main")
        desktop_content = f"""[Desktop Entry]
Version=1.0
Type=Application
NameЛабораторная= 2 (Trial)
CommentПрограмма= с контролем запусков
Exec={exec_path}
Icon=system-lock-screen
Terminal=false
Categories=Utility;Application;
StartupNotify=true
"""
        # Создание пункта в меню приложений
        with open(MENU_DESKTOP_FILE, "w", encoding="utf-8") as f:
            f.write(desktop_content)
        os.chmod(MENU_DESKTOP_FILE, 0o755)

        # Создание ярлыка на Рабочем столе
        with open(DESKTOP_SHORTCUT_FILE, "w", encoding="utf-8") as f:
            f.write(desktop_content)
        os.chmod(DESKTOP_SHORTCUT_FILE, 0o755)

        # Установка доверенного атрибута запуска для среды KDE/GNOME
        try:
            subprocess.run(["gio", "set", DESKTOP_SHORTCUT_FILE, "metadata::
    trusted", "true"], stderr=subprocess.DEVNULL)
```

```
        except Exception:
            pass

        messagebox.showinfo(
            "Успех",
            "Программа успешно установлена!\n"
            "• Добавлена в меню KDE\n"
            "• Ярлык создан на Рабочем столе"
        )
        root.destroy()
    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось установить: {e}")

root = Tk()
root.title("Установка Лабораторной №2")
root.geometry("460x230")
root.resizable(False, False)

was_installed, launches = check_previous_install()

if was_installed:
    lbl_status = Label(
        root,
        text=f"Внимание!\Программаn УЖЕ была установлена на этом ПК.\
   Использованоn запусков ранее: {launches}.\Лимитыn не будут сброшены!",
        fg="darkorange",
        font=("sans-serif", 10, "bold")
    )
else:
    lbl_status = Label(
        root,
        text="Первичная установка программы в систему KDE.\Будутn созданы
   ярлыки в меню и на рабочем столе.",
        fg="green"
    )

lbl_status.pack(pady=15)
btn_install = Button(root, text="Установить", width=22, height=2, command=
   install)
btn_install.pack(pady=10)

root.mainloop()
```

Листинг 2. Основная рабочая программа (main.py)

```
import datetime
import os
import subprocess
from tkinter import Button, Entry, Label, Tk, messagebox
import webbrowser

HOME_PATH = os.path.expanduser("~/.config/lab2")
CONF_FILE = "lab2.txt"
CONF_PATH = f"{HOME_PATH}/{CONF_FILE}"

INSTALL_DIR = os.path.expanduser("~/.local/share/lab2")

# Конфигурация ограничений ознакомительного периода
MAX_LAUNCHES = 3
MAX_LAUNCHES_DELTA = datetime.timedelta(minutes=1)

def is_trial_expired(launches: int, first_launch: datetime.datetime) -> bool:
    """Проверка наступления условий блокировки по счетчику или времени"""
    now = datetime.datetime.now(tz=datetime.timezone.utc)
    return (launches >= MAX_LAUNCHES) or (now - MAX_LAUNCHES_DELTA >
   first_launch)

def run_uninstaller():
    """Вызов деинсталлятора при блокировке программы"""
    possible_paths = [
        os.path.join(INSTALL_DIR, "uninstaller"),
        os.path.join(INSTALL_DIR, "uninstaller.py"),
        os.path.join(os.path.dirname(os.path.abspath(__file__)), "uninstaller"),
        os.path.join(os.path.dirname(os.path.abspath(__file__)), "uninstaller.py
   ")
    ]

    uninstaller_path = None
    for path in possible_paths:
        if os.path.exists(path):
            uninstaller_path = path
            break

    if uninstaller_path:
        if uninstaller_path.endswith(".py"):
            subprocess.Popen(["python3", uninstaller_path])
        else:
            subprocess.Popen([uninstaller_path])
        root.destroy()
    else:
        messagebox.showerror("Ошибка", "Деинсталлятор не найден в системе!")
```

```
def show_error():
    """Переключение графического интерфейса в режим блокировки"""
    global entry, label_error, label_info, btn_uninstall

    error_message = (
        "Вы превысили лимит запусков программы"
        if conf["launches"] >= MAX_LAUNCHES
        else "Вы превысили лимит времени использования программы"
    )

    label_error = Label(text=error_message, fg="red", font=("sans-serif", 9, "
    bold"))
    label_error.pack(pady=3)

    entry.config(state="disabled")
    button.config(state="disabled")

    label_info.config(
        text="Приобрести лицензию можно у @I_WOOPER_I",
        fg="blue",
        cursor="hand2"
    )
    label_info.bind("<Button-1>", lambda e: webbrowser.open("https://t.me/
    I_WOOPER_I"))

    btn_uninstall = Button(
        root,
        text="Удалить программу Деинсталляция()",
        fg="red",
        command=run_uninstaller
    )
    btn_uninstall.pack(pady=5)

def read_conf():
    """Считывание конфигурации и первичная инициализация при первом старте"""
    with open(CONF_PATH, "r") as f:
        conf = f.read().split("\n")

    now = datetime.datetime.now(tz=datetime.timezone.utc)
    if len(conf) >= 3 and len(conf[1]) > 0 and len(conf[2]) > 0:
        new_conf = {
            "name": conf[0],
            "launches": int(conf[1]),
            "first_launch": datetime.datetime.fromisoformat(conf[2])
        }
    else:
        new_conf = {
            "name": "",
            "launches": 0,
```

```
            "first_launch": now
        }
    return new_conf

def save_conf():
    """Инкремент счетчика и сохранение ФИО в файл"""
    global entry, conf, label_error, label_info

    if is_trial_expired(conf["launches"], conf["first_launch"]):
        show_error()
        return

    conf["name"] = entry.get() or ""
    conf["launches"] = conf.get("launches", 0) + 1

    with open(CONF_PATH, "w") as f:
        f.write("\n".join(str(v) for v in conf.values()))

    label_info.config(
        text="Данные сохранены!\n" + f"Бесплатных запусков осталось: {
    MAX_LAUNCHES - conf['launches']}"
    )

    if is_trial_expired(conf["launches"], conf["first_launch"]):
        show_error()

if not os.path.exists(CONF_PATH):
    os.makedirs(HOME_PATH, exist_ok=True)
    with open(CONF_PATH, "w") as f:
        f.write("")
conf = read_conf()

root = Tk()
root.title("Trial Program")
root.geometry("450x290")
root.resizable(False, False)

label_name = Label(
    text=f"Ваше ФИО: {conf['name']}\Изменитьn:" if len(conf["name"]) > 0 else "
    Введите свое ФИО"
)
label_name.pack(pady=10)

entry = Entry()
entry.pack()

button = Button(text="Сохранить", command=save_conf)
button.pack(pady=10)
```

```
remaining = MAX_LAUNCHES - conf["launches"]
label_info = Label(
    text=f"Бесплатных запусков осталось: {remaining}" if not is_trial_expired(
    conf["launches"], conf["first_launch"]) else ""
)
label_info.pack(pady=5)

if is_trial_expired(conf["launches"], conf["first_launch"]):
    show_error()

root.mainloop()
```

Листинг 3. Модуль деинсталляции (uninstaller.py)

```
import os
import shutil
import subprocess
from tkinter import Tk, Label, Button, messagebox

def get_desktop_dir():
    """Получение папки Desktop согласно стандарту XDG"""
    try:
        res = subprocess.check_output(["xdg-user-dir", "DESKTOP"], text=True).
    strip()
        if os.path.exists(res):
            return res
    except Exception:
        pass
    for fallback in [os.path.expanduser("~/Desktop"), os.path.expanduser("
    Рабочий~/ стол")]:
        if os.path.exists(fallback):
            return fallback
    return os.path.expanduser("~/Desktop")

HOME = os.path.expanduser("~")
INSTALL_DIR = os.path.join(HOME, ".local", "share", "lab2")
APPS_DIR = os.path.join(HOME, ".local", "share", "applications")
DESKTOP_DIR = get_desktop_dir()

MENU_DESKTOP_FILE = os.path.join(APPS_DIR, "lab2.desktop")
DESKTOP_SHORTCUT_FILE = os.path.join(DESKTOP_DIR, "lab2.desktop")

def uninstall():
    """Удаление исполняемых компонентов без удаления конфигурации лицензии"""
    confirm = messagebox.askyesno("Удаление", "Вы действительно хотите удалить
    программу?")
    if not confirm:
        return

    try:
        # 1. Удаление пункта в меню приложений KDE
        if os.path.exists(MENU_DESKTOP_FILE):
            os.remove(MENU_DESKTOP_FILE)

        # 2. Удаление ярлыка с Рабочего стола
        if os.path.exists(DESKTOP_SHORTCUT_FILE):
            os.remove(DESKTOP_SHORTCUT_FILE)

        # 3. Полное удаление директории установленного бинарного ПО
        if os.path.exists(INSTALL_DIR):
            shutil.rmtree(INSTALL_DIR)
```

```
        # 4. Обновление баз данных и сброс кэша рабочего окружения KDE
        subprocess.run(["update-desktop-database", APPS_DIR], stderr=subprocess.
    DEVNULL, stdout=subprocess.DEVNULL)
        for cmd in ["kbuildsycoca6", "kbuildsycoca5"]:
            try:
                subprocess.run([cmd, "--noincremental"], stderr=subprocess.
    DEVNULL, stdout=subprocess.DEVNULL)
                break
            except FileNotFoundError:
                continue

        # Лицензионный маркер ~/.config/lab2/lab2.txt намеренно НЕ удаляется!

        messagebox.showinfo("Успешно", "Программа и ярлыки удалены с компьютера!
    ")
        root.destroy()
    except Exception as e:
        messagebox.showerror("Ошибка", f"Ошибка при удалении: {e}")

root = Tk()
root.title("Деинсталлятор")
root.geometry("400x180")
root.resizable(False, False)

lbl = Label(root, text="Мастер удаления программы Лабораторной №2.", pady=15)
lbl.pack()

btn = Button(root, text="Деинсталлировать", fg="red", width=20, height=2,
    command=uninstall)
btn.pack(pady=10)

root.mainloop()
```